Architecture

The `apps/web` package is the reviewer-facing front end of the accounts-payable platform — a single-page React application where users manage vendors, upload invoices and quotations, and review the line items the backend extracts and reconciles against agreed rate cards.

It lives in a pnpm monorepo alongside the document-processing service (`apps/api`) and the shared UI library (`packages/ui`). The two apps divide the work: the service runs extraction, matching, and reconciliation, and the web app owns the reviewer-facing side — driving that pipeline over HTTP, holding the resulting server state, and turning it into a workflow a person reviews, judges, and signs off. Data ownership lives in the service; interaction, state, and review flow live here.

Stack

Layer	Technology	Role
Framework	React 19 + Vite + TypeScript	SPA with the React Compiler enabled (auto-memoization)
Routing	TanStack Router (file-based)	Typed routes, route guards, per-route `staticData`
Server state	TanStack Query	Queries, mutations, cache invalidation
HTTP	axios (`src/lib/api.ts`)	`withCredentials` session cookie; 401 interceptor
Auth	better-auth	Session-cookie auth; server manages refresh
UI	`@nom/ui` (shadcn) + Tailwind v4	Shared component library in `packages/ui`
Forms	React Hook Form + Zod	`zodResolver` ; composition over boolean-mode props
Tables	TanStack Table	Per-feature inline `ColumnDef[]` , no generic wrapper
Local state	Zustand	Small UI state (dialog open-state, session-expired)
Lint/format	Biome	2-space, no ESLint/Prettier

System Boundaries

- `src/routes/` — file-based route tree (generated `routeTree.gen.ts` , never hand-edited). `_app.tsx` is the protected pathless layout (session guard + app shell); `(auth)/` is the public group.
- `src/features/<feature>/` — feature code, organized by kind: `components/` , `services/` (queries/mutations + mappers), `hooks/` , `lib/` (pure helpers), `store/` (Zustand), `schemas.ts` , `types.ts` . A folder appears with its first file of that kind — no empty scaffolding.
- `src/shared/` — helpers/hooks/components reused **across** features (a feature-local one graduates here the moment a second feature uses it).
- `src/lib/` — app-wide infrastructure: the axios client, env, error helpers.
- `src/stores/` — truly global Zustand stores (e.g. session-expired).

- `packages/ui/` — the shared shadcn primitives (imported via `@nom/ui/*`).

Features: `auth`, `vendors`, `quotations`, `invoices`, `rules`, `dashboard`.

Data Layer

- **Server state is TanStack Query.** Each feature defines `queryOptions` and mutation hooks in `src/features/<feature>/services/`. Mappers that reshape an API payload live beside them (they are data-layer, not generic helpers).
- **HTTP** goes through the single axios instance. The API wraps responses as `{ success, data, ... }` — unwrap to `res.data.data`. A 401 opens the session-expired dialog via the response interceptor.
- **Mutations invalidate** (`qc.invalidateQueries`). There are no optimistic mutations: every destructive action sits behind a confirm dialog, which is itself the "action registered" feedback, so an optimistic write only desynchronised the row's removal from the dialog closing and the toast. Errors surface via `toast.error(getErrorMessage(err, "..."))`.
- **Types mirror the wire.** Feature `types.ts` files are hand-written local mirrors of the backend DTOs, narrowed to the fields the UI actually reads (see `features/invoices/types.ts`). They are not imported from the backend.

The Invoice Pipeline (client orchestration)

The web app drives a multi-step backend pipeline; the UI reflects each step's status rather than computing it. The work is split across two pages:

```
/invoice/add     upload → confirm-many → extract-metadata → validate → extract-line-items
/invoice/review  match-line-items (auto, on arrival) → poll match-status if queued → review
```

- The upload saga lives in `features/invoices/hooks/use-invoice-submit.ts`. Each stage fans out across the batch with `Promise.all`, so the steps are sequential but the documents within a step are not.
- **Matching is not part of the upload saga.** It fires from an effect when the review page mounts (`hooks/use-document-matcher.ts`), once per document per session — guarded by a ref rather than by status, because a synchronous match never flips the document to `matched`. This is the one place the invoice flow diverges from quotations, which match at upload.
- `match-line-items` returns either 201 (synchronous, with match counts) or 202 (queued) — in the queued case `matchStatusQuery` polls `match-status` every 2s while the status reads `matching`.
- Two user-facing gates can halt the saga: **invalid files** (validate answers 200 with `status: "rejected"`, not a 4xx) and **validation warnings**, where the reviewer chooses continue-or-discard.
- Document status (`pending → extracted → validated → processing → committed → matching → matched`) is condensed to two tones by `statusTone` (`features/invoices/lib/invoice-status.ts`). It is **not** shown as a column — it only drives the list's client-side status filter.
- Review status (`Pending` / `Completed`) is separate, derived server-side from a `reviewCompleted` flag, and is what the list's Status column renders. The client sets it via `mark-reviewed`, POSTed for every document in the batch when the review footer's "Complete" is confirmed.

Conventions

- **Direct imports only** — no barrel files.
- **Composition over configuration** — explicit variant components (`AddVendorForm` / `EditVendorForm`) sharing a fields body, not `isEdit` flags or `renderX` props.
- **No manual `useMemo` / `useCallback`** — the React Compiler auto-memoizes. Compute inline; follow the Rules of React so it can optimize.
- **Reuse** `cn()` , `getErrorMessage` , the sonner `toast` , and shared components (e.g. `TruncatedText`) rather than re-implementing.
- **Comments** default to none; when the *why* is non-obvious, write a JSDoc block above the symbol, not an inline `//` .

Invariants

1. No business logic in the client — extraction, matching, and reconciliation are server-side. The web app orchestrates and renders.
2. All server state flows through TanStack Query; components do not call axios directly outside `services/` .
3. Every mutation invalidates its affected queries — no stale reads after a write.
4. Auth is enforced at the route boundary (`_app` `beforeLoad`); in-page 401s only surface the session-expired dialog.
5. `routeTree.gen.ts` is generated — routes change by adding/moving files and re-running the dev server, never by editing the generated tree.
6. Feature `types.ts` mirror the backend DTOs; when a DTO changes, update the mirror rather than importing across the app boundary.

Getting Started

1. Install workspace deps from the repo root: `pnpm install` .
2. Copy `apps/web/.env.example` to `apps/web/.env` and point `VITE_API_URL` at the running API — it must include the version prefix, e.g. `http://localhost:8000/api/v1` . This is required: `src/lib/env.ts` validates it at startup and the app throws if it's missing or not a URL.
3. Start the backend (`apps/api`) — auth is a session cookie, so the API must be running and reachable at `VITE_API_URL` or every page redirects to login.
4. Run the dev server: `pnpm --filter web dev` .

Commands (from repo root)

```
pnpm --filter web dev          # Vite dev server (regenerates the route tree)
pnpm --filter web typecheck    # tsc --noEmit
pnpm --filter web lint         # biome lint
pnpm --filter web build        # tsc -b && vite build
PUSH_FILTER=web git push       # scope the pre-push hook to the frontend
```